



PRÉ-RENTRÉE 2026-2027

SÉANCE 2 - BOUCLES, COMPTEURS ET ACCUMULATEURS

Dossier personnalisé - Ahmad BELDI - durée : 2 heures

Parcours : Fondations guidées

Objectif personnel de la séance 2 : comprendre exactement ce que produit `range`, distinguer un `compteur` d'un `accumulateur`, puis écrire une boucle qui calcule une somme **sans erreur de borne**.

Production à conserver : le fichier `S2_boucles_Ahmad.py`, capable d'analyser une liste de mesures : somme, effectif, moyenne, maximum et nombre de valeurs dépassant un seuil.

Ma méthode pour cette séance

1. Avant d'exécuter, j'écris les valeurs produites par `range`.
2. Je trace les variables tour après tour.
3. J'initialise compteur, somme ou maximum **avant** la boucle.
4. Je prévois au moins un résultat à la main, puis je compare.
5. Je ne copie pas un code que je ne peux pas expliquer ligne par ligne.

1. Feuille de route des 120 minutes

Temps	Travail	Preuve attendue
0–10 min	Rituel sans ordinateur : <code>range</code> et somme cumulée.	Deux prévisions justifiées.
10–30 min	Bornes, pas, sens de parcours et indices.	Valeurs de quatre <code>range</code> exactes.
30–50 min	Compteur, accumulateur et maximum.	Table de trace complète.
50–65 min	Choisir <code>for</code> ou <code>while</code> .	Choix justifié et boucle qui termine.
65–70 min	Pause.	–
70–100 min	Atelier principal : analyser une série de mesures.	Programme exécutable et commenté.
100–112 min	Parcours Ahmad : somme de 1 à <code>n</code> et correction d'un bug.	Boucle sans erreur de borne.
112–120 min	Invariant simple, contrôle et exit ticket.	Réponses individuelles.

Je saurai que j'ai progressé si : je peux écrire les valeurs d'un `range` sans exécuter, expliquer la différence entre `compteur += 1` et `total += valeur`, et obtenir les résultats attendus sur au moins trois jeux de test.

2. Rituel et diagnostic flash - 10 min

Fichier à créer : S2_boucles_Ahmad.py. Commencer par :

```
print("Nexus NSI - seance 2")
```

Rituel 1 - valeurs produites

Sans ordinateur, écrire dans l'ordre toutes les valeurs produites par :

```
range(2, 9, 3)
```

Certitude : ☐1 ☐2 ☐3 ☐4

Contrôle : _____

Rituel 2 - tracer une somme

Tracer le programme sans l'exécuter.

```
s = 0
for i in range(4):
    s = s + i
```

Tour	i	s avant	s après	Calcul effectué
1				
2				
3				
4				

Valeur finale de **s** : _____

Nombre de tours : _____

Certitude : ☐1 ☐2 ☐3 ☐4

Contrôle : _____

Attention. Le nombre de tours et la somme obtenue sont deux informations différentes. Le compteur répond à « combien ? » ; l'accumulateur répond ici à « quelle somme ? ».

Rappel de la séance 1

Dans **x = x + 1**, Python calcule d'abord _____, puis _____.

3. Comprendre range - 20 min

Repère. Dans `range(debut, fin, pas)`, la valeur `debut` est incluse et la valeur `fin` est exclue. Le `pas` indique comment on passe d'une valeur à la suivante.

A. Bandes range - compléter avant exécution

Expression	Première valeur	Dernière valeur	Nombre de tours	Valeurs produites
<code>range(5)</code>				
<code>range(2, 8)</code>				
<code>range(10, 2, -2)</code>				
<code>range(5, 5)</code>				

B. Construire le bon range

Écrire un `range` qui produit exactement :

- 1, 2, 3, 4, 5 : _____
- 0, 1, 2, ..., n-1 : _____
- 2, 4, 6, 8, 10 : _____
- 9, 6, 3 : _____

C. Erreur de borne à corriger

Le programme veut afficher les entiers de 1 à 5 inclus.

```
for i in range(1, 5):  
    print(i)
```

Valeur oubliée : _____ Correction : _____

Checkpoint range. Nombre de séries écrites sans erreur : _____ / 8. Je peux expliquer pourquoi la borne de fin est exclue : ☐oui ☐pas encore.

Aides graduées - une seule à la fois : A - écrire les valeurs de `range` ; B - nommer le schéma (compter, additionner, chercher, répéter) ; C - initialiser avant la boucle ; D - vérifier ce qui évolue ; E - compter les tours et comparer avec un calcul manuel.

4. Compteur, accumulateur et maximum - 20 min

Schéma	Question à laquelle il répond	Initialisation fréquente
Compteur	Combien de valeurs vérifient une propriété ?	<code>compteur = 0</code>
Accumulateur	Quelle somme construit-on progressivement ?	<code>total = 0</code>
Maximum	Quelle est la plus grande valeur déjà lue ?	premier élément d'une liste non vide

On utilise la liste :

```
mesures = [12, -3, 7, 0, 15, 9]
```

A. Compléter le code

```
total = _____
nb_positives = _____

for valeur in mesures:
    total = _____
    if _____:
        nb_positives = _____
```

B. Tracer les deux états

Tour	valeur	total avant	total après	nb avant	nb après
1					
2					
3					
4					
5					
6					

C. Comprendre les rôles

1. Pourquoi `total` et `nb_positives` doivent-ils être initialisés avant la boucle ?

2. Quelle différence y a-t-il entre `total += valeur` et `nb_positives += 1` ?

3. Pour calculer un maximum sur une liste non vide, pourquoi `maximum = mesures[0]` est-il plus sûr que `maximum = 0` ?

5. Choisir entre for et while - 15 min

Idée centrale. `for` convient à un parcours borné ou à une collection connue. `while` convient lorsqu'on répète tant qu'une condition est vraie. Dans une boucle `while`, une donnée de la condition doit évoluer vers l'arrêt.

A. Classer et justifier

Situation	for / while ?	Justification
Parcourir toutes les valeurs d'une liste		
Afficher exactement 10 nombres		
Demander un mot de passe jusqu'à ce qu'il soit correct		
Lire des valeurs jusqu'au mot <code>fin</code>		
Parcourir les caractères d'un texte		

B. Boucle qui ne termine pas

```
n = 1
while n <= 5:
    print(n)
```

1. Quelle est la condition ? _____
2. Quelle variable devrait évoluer ? _____
3. Ajouter une seule instruction pour obtenir 1, 2, 3, 4, 5, puis l'arrêt.

```
n = 1
while n <= 5:
    print(n)
```

4. Expliquer pourquoi la version corrigée termine.

Contrôle manuel. Avant l'exécution, écrire les cinq valeurs successives de `n` :

Aides graduées - une seule à la fois : A - écrire les valeurs de `range` ; B - nommer le schéma (compter, additionner, chercher, répéter) ; C - initialiser avant la boucle ; D - vérifier ce qui évolue ; E - compter les tours et comparer avec un calcul manuel.

6. Atelier principal - analyser une série - 30 min

Spécification. À partir d'une liste non vide de mesures et d'un seuil, le programme doit calculer :

- la somme des mesures ;
- l'effectif ;
- la moyenne ;
- le maximum ;
- le nombre de mesures supérieures ou égales au seuil.

Données de travail :

```
mesures = [18, 21, -2, 27, 31, 15]
seuil = 25
```

Étape 1 - résultats attendus AVANT le code

Grandeur	Calcul manuel ou raisonnement	Résultat attendu
Effectif		
Somme		
Moyenne		
Maximum		
Nombre de valeurs ≥ 25		

Étape 2 - choisir les variables

Variable	Rôle	Valeur initiale
total		
effectif		
maximum		
nb_alerte		

Étape 3 - plan en français

Compléter les phrases :

1. Pour chaque _____ de la liste, je l'ajoute à _____.
2. À chaque tour, j'augmente _____ de 1.
3. Si la valeur dépasse le seuil, j'augmente _____.
4. Si la valeur dépasse le maximum courant, _____.
5. Après la boucle, je calcule la moyenne en divisant _____ par _____.

7. Atelier principal - coder et expliquer

Compléter, exécuter, puis corriger une seule chose à la fois.

```
mesures = [18, 21, -2, 27, 31, 15]
seuil = 25

total = _____
effectif = _____
nb_alerte = _____
maximum = _____

for valeur in mesures:
    total = _____
    effectif = _____

    if _____:
        nb_alerte = _____

    if _____:
        maximum = _____

moyenne = _____

print("somme :", total)
print("effectif :", effectif)
print("moyenne :", moyenne)
print("maximum :", maximum)
print("mesures en alerte :", nb_alerte)
```

Mes trois checkpoints

1. Après le premier tour, quelles sont les valeurs de `total`, `effectif`, `maximum` et `nb_alerte` ?

2. Après les six tours, que doit valoir `effectif` ? Comparer avec `len(mesures)`.

3. Expliquer le rôle de `maximum` sans lire le code mot à mot.

Production à sauvegarder : `S2_boucles_Ahmad.py`. Le fichier doit s'exécuter sans erreur et afficher les cinq informations demandées.

8. Parcours personnalisé Ahmad - 12 min

Défi A - somme de 1 à n

On suppose $n \geq 1$. Compléter le programme, puis tester $n = 1$, $n = 5$ et $n = 10$.

```
n = 5
total = _____

for i in range(_____):
    total = _____

print(total)
```

Valeurs produites par le range pour $n = 5$: _____

Contrôle manuel : comparer au calcul $1 + 2 + \dots + n$. Résultat prévu pour $n = 5$: _____

Défi B - compteur réinitialisé au mauvais endroit

```
valeurs = [-2, 4, 7, -1, 3]

for valeur in valeurs:
    nb_positives = 0
    if valeur > 0:
        nb_positives += 1

print(nb_positives)
```

1. Prévoir la sortie actuelle : _____
2. Entourer l'instruction placée au mauvais endroit.
3. Réécrire uniquement les lignes nécessaires.

```
# Version corrigee
```

4. Expliquer pourquoi la correction fonctionne.

Défi C - erreur de borne

Pour afficher les indices valides de $L = [8, 3, 6]$, choisir puis justifier :

`range(len(L))` ou `range(len(L) + 1)`

9. Tests, débogage et invariant - 10 min

A. Tests prévus avant la dernière exécution

Test	Entrée	Résultat attendu	Résultat obtenu	OK
1	[18, 21, -2, 27, 31, 15]			☐
2	[5]			☐
3	[-4, -9, -2]			☐
4	Une liste inventée			☐

B. Journal de débogage

Symptôme observé	Hypothèse	Modification testée	Résultat

C. Mon premier invariant

Compléter avec tes mots :

Après k tours de la boucle :

- `effectif` vaut _____ ;
- `total` contient _____ ;
- `nb_alerte` compte _____ ;
- `maximum` est _____.

Aides graduées - une seule à la fois : A - écrire les valeurs de `range` ; B - nommer le schéma (compter, additionner, chercher, répéter) ; C - initialiser avant la boucle ; D - vérifier ce qui évolue ; E - compter les tours et comparer avec un calcul manuel.

10. Trace écrite, exit ticket et auto-évaluation - 8 min

Trace écrite à conserver

- `range(debut, fin, pas)` inclut le début et exclut la fin.
- Un **compteur** augmente généralement de 1 ; un **accumulateur** ajoute les valeurs rencontrées.
- Les variables d'état sont initialisées avant la boucle.
- **for** convient à un parcours connu ; **while** exige une condition qui évolue vers l'arrêt.
- Une réponse sans prévision ni test reste une hypothèse.

Exit ticket - sans aide

1. Écrire les valeurs produites par `range(1, 8, 2)`.

Certitude : ☐ 1 ☐ 2 ☐ 3 ☐ 4

Contrôle : _____

2. Expliquer la différence entre `compteur += 1` et `total += valeur`.

3. Pour répéter une saisie tant qu'elle est invalide, choisir **for** ou **while** et justifier.

4. Où doit être placée l'initialisation d'un compteur ? Pourquoi ?

Mon auto-évaluation

Je peux...	Pas encore	Avec aide	Seul	Expliquer
Écrire les valeurs d'un range	☐	☐	☐	☐
Choisir entre for et while	☐	☐	☐	☐
Utiliser compteur et accumulateur	☐	☐	☐	☐
Tracer une boucle sans ordinateur	☐	☐	☐	☐
Tester et corriger une erreur de borne	☐	☐	☐	☐

Mon mini-plan après S2

Une boucle que je referai sans aide : _____

Une erreur que je saurai reconnaître : _____

Une preuve de progrès que je conserve : _____